

Code 1:

Input: Live camera frames obtained through OpenCV's VideoCapture(0) function.

output:

A real-time video display window named “**camera stream**” showing the webcam feed.

Logic:

1. Initialize camera: cv2.VideoCapture(0) opens the default webcam (index 0).
2. Read frames in a loop: cap.read() captures each frame from the webcam.
ret → True if frame is successfully captured.
frame → the actual image captured.
3. Check capture success: If ret is False, exit the loop.
4. Display frame: cv2.imshow() shows the current frame in a window.
5. Exit condition: cv2.waitKey(1) & 0xFF == ord("q") waits 1 ms for key press; stops if "q" is pressed.
6. Release resources: cap.release() closes the camera; cv2.destroyAllWindows() closes the display window.

Code 2:

Input: The input source for this code is the webcam feed accessed through cv2.VideoCapture(0).

output:

This code outputs real-time video on the screen and generates a collection of frame images stored inside the ‘frames’ folder.

Logic:

1. Initialize camera: cv2.VideoCapture(0) opens the default webcam.
2. Create folder: os.makedirs("frames", exist_ok=True) ensures a folder named frames exists to save images.
3. Frame counter: frame_count = 0 keeps track of the number of frames captured.
4. Capture loop:
cap.read() grabs a frame from the webcam.

If capture fails (`ret=False`), exit the loop.

Display the frame with `cv2.imshow()`.

Save the frame as a JPEG in `frames/` folder using a zero-padded filename: `frame_000000.jpg`, `frame_000001.jpg`, ...

Increment `frame_count`.

5. Exit on key press: Press "q" to stop capturing.

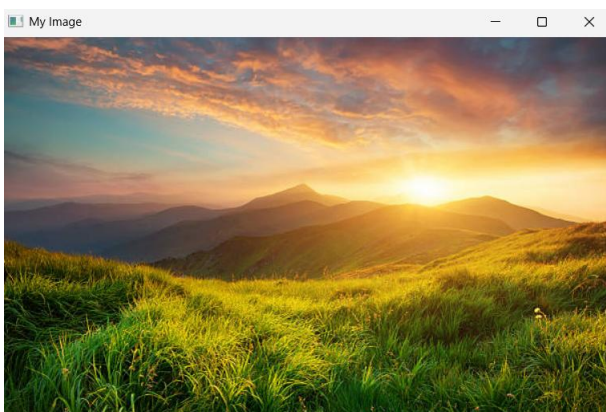
6. Release resources: Close the camera and window using `cap.release()` and `cv2.destroyAllWindows()`.

Code 3:

Input:



Output:



Logic:

1. Read the image from the specified path using `cv2.imread()`.
2. Check if the image was loaded (img is None).
 - If not, print an error message.
3. If the image is loaded, display it in a window using `cv2.imshow()`.
4. Wait for a key press using `cv2.waitKey(0)`.
5. Close the display window with `cv2.destroyAllWindows()`.

Code 4:

input:



Output:



original



Flipped Vertically



Flipped Horizontally



Flipped Both

Logic:

1. **Read the image** from the file path.
2. **Check** if the image loaded correctly.
3. **Flip the image** in different ways using `cv2.flip()`:
 - 0 → Flip **vertically**
 - 1 → Flip **horizontally**
 - -1 → Flip **both vertically and horizontally**
4. **Display** the original and flipped images in separate windows.
5. **Wait** for a key press and **close** all windows.

Code 5:

Input:



Output:



original



Resized

Logic:

1. Load Image: `cv2.imread()` reads the image from the given file path.
2. Check if Loaded: If the image is not found, show an error and stop the program.
3. Resize Image: `cv2.resize()` changes the image size to 300×300 pixels.

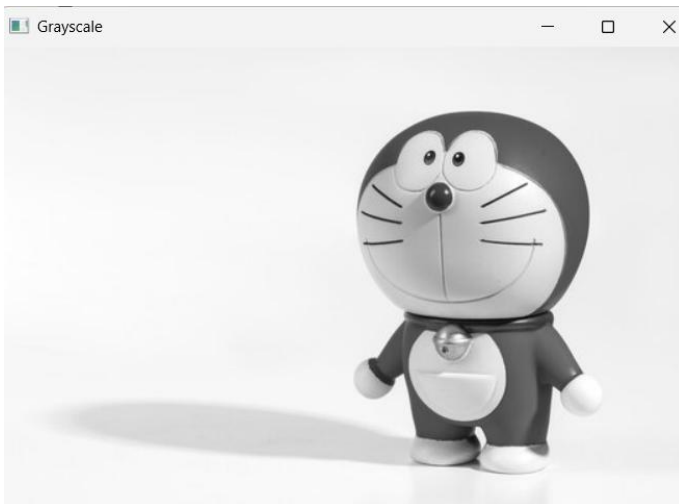
4. Display Images: Shows both the original and resized images in separate windows.
5. Wait for Key Press: `cv2.waitKey(0)` waits until a key is pressed.
6. Close Windows: `cv2.destroyAllWindows()` closes image display windows.
7. Save Resized Image: `cv2.imwrite()` saves the resized image as `resized_output.jpg`.

Code 6:

Input:



Output:



Logic:

1. Read an image from the given file path.
2. Check whether the image loaded correctly.

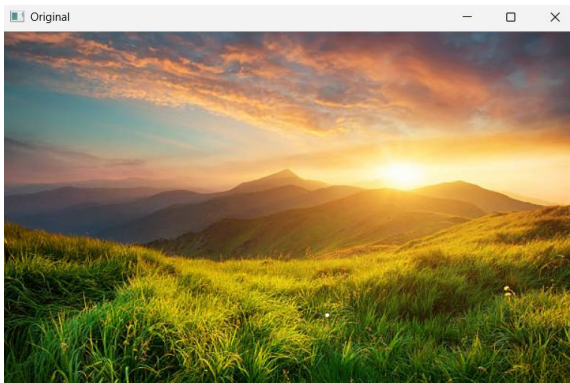
3. Convert the loaded color image into a grayscale image.
4. Display both the original and grayscale images.
5. Wait for a key press and then close the windows.
6. Save the grayscale image as an output file.

Code 7:

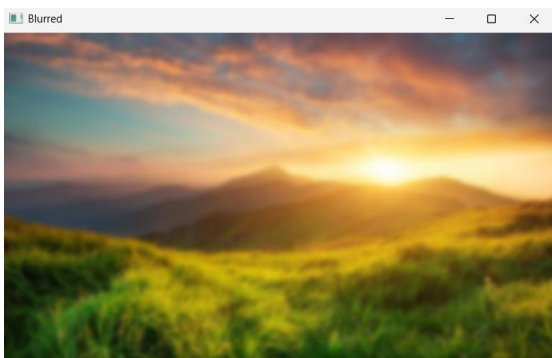
Input:



Output:



Original



Blurred

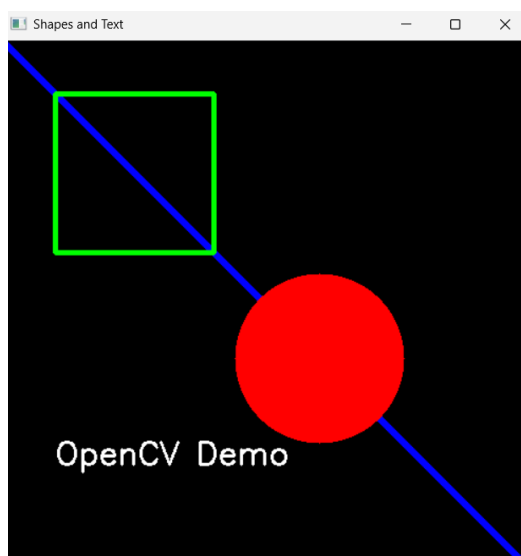
Logic:

1. `cv2.imread()` → Loads the image from your computer.
If the image is not found, it prints an error and stops.
2. `cv2.GaussianBlur(img, (15, 15), 0)` →
Applies a *Gaussian blur* (smoothing effect) on the image.
 - (15, 15) is the blur kernel size (bigger → more blur).
 - 0 is standard deviation (auto-calculated).
3. `cv2.imshow()` → Displays the original image and the blurred image in two separate windows.
4. `cv2.waitKey(0)` → Keeps the windows open until you press any key.
5. `cv2.destroyAllWindows()` → Closes the display windows.
6. `cv2.imwrite("blurred_output.jpg", blur)` → Saves the blurred image to your computer with the name *blurred_output.jpg*.

code 8:

Input: The input for this program is a blank 500×500 image created using NumPy.

Output:



Logic:

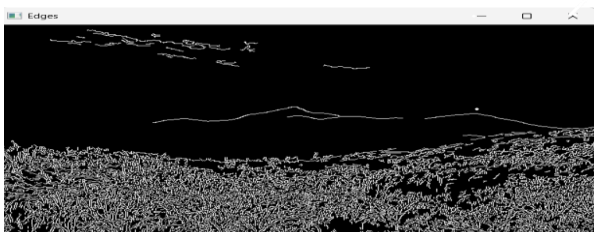
1. `np.zeros((500, 500, 3))`
Creates a blank black image of size 500×500 pixels with 3 color channels (RGB).
2. `cv2.line(...)`
Draws a blue line from the top-left corner (0,0) to the bottom-right (500,500).
5 → thickness of the line.
3. `cv2.rectangle(...)`
Draws a green rectangle with corners at (50,50) and (200,200).
3 → outline thickness.
4. `cv2.circle(...)`
Draws a filled red circle centered at (300,300) with radius 80.
-1 → means filled circle.
5. `cv2.putText(...)`
Writes the text "OpenCV Demo" on the image at position (50, 400).
Text is in white with thickness 2.
6. `cv2.imshow(...)`
Displays the final image with shapes and text.
7. `cv2.waitKey(0)` & `cv2.destroyAllWindows()`
Keeps the window open until a key is pressed, then closes it.

Code 9:

Input:



Output:



Logic: This code performs edge detection on an image using OpenCv's canny Algorithm.

1. `cv2.imread` Reads the image in grayscale mode (the argument 0 specifies grayscale). Grayscale conversion simplifies processing since edge detection works on intensity changes rather than colors.

2. `cv2.Canny(img, 100, 200)`

Applies the Canny Edge Detection algorithm on the grayscale image.

3. `cv2.imshow("Edges", edges)`

Displays the output image showing the detected edges.

4. `cv2.waitKey(0)`

Waits indefinitely until any key is pressed.

5. `cv2.destroyAllWindows()`

Closes all opened image windows.

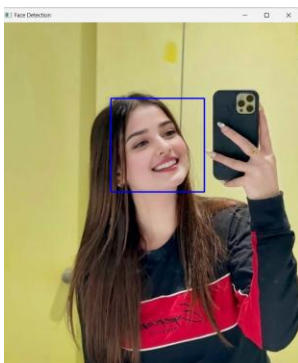
Purpose: The program reads an image, converts it to grayscale, applies Canny edge detection, and displays the detected edges. This helps highlight object boundaries and is commonly used in computer vision tasks like shape detection and object recognition.

Code 10:

Input:



Output:



Face Detection

Logic:

1. Loads the pre-trained Haar Cascade classifier for frontal faces.
2. Reads the image and converts it to grayscale (face detection works better on grayscale).
3. Uses detectMultiScale() to detect faces, returning coordinates and sizes of bounding boxes.
4. Draws rectangles around detected faces on the original image.
5. Displays the resulting image with faces highlighted.

Code 11:**Input:****Output:**

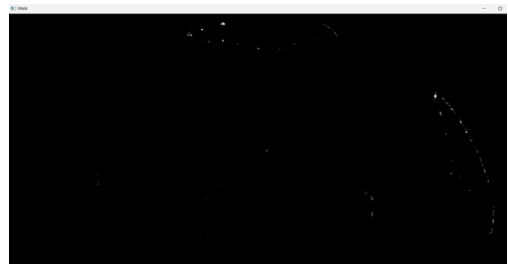
Logic:

This code detects and outlines object boundaries (contours) in an image.

1. Reads the image and converts it to grayscale.
2. Applies binary thresholding to separate objects from the background.
3. Finds contours (object outlines) from the thresholded image.
4. Draws all contours on the original image in green color with thickness 2.
5. Displays the final image with contours highlighted.

Code 12:**Input:****Output:**

original



mask



Filtered

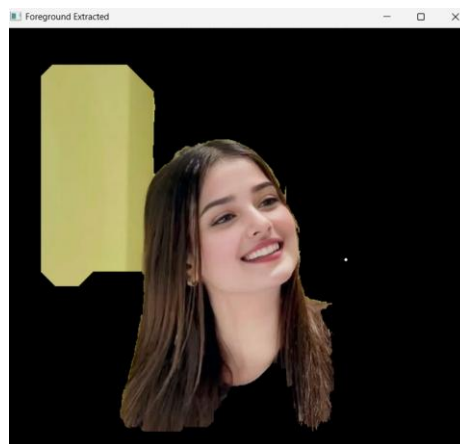
Logic:

1. Reads the image and converts it to HSV color space (better for color detection).
2. Defines the HSV range for blue color.
3. Creates a mask that highlights only blue areas.
4. Uses `bitwise_and()` to extract and display only the blue-colored parts of the image.
5. Shows the original image, the mask, and the filtered (blue-only) result.

Purpose: To detect and isolate blue objects or regions in an image using color filtering.

Code 13:**Input:****Output:**

Original



Foreground Extracted

Logic:

1. Reads the image and creates an initial mask (all zeros).
2. Defines a rectangle (ROI) around the main object.
3. Uses GrabCut, an iterative segmentation algorithm, to separate the foreground (main object) from the background using color and edge information.
4. Updates the mask to keep only the foreground pixels.
5. Multiplies the mask with the original image to display only the extracted object, removing the background.
6. Shows the original image and the foreground-extracted result.

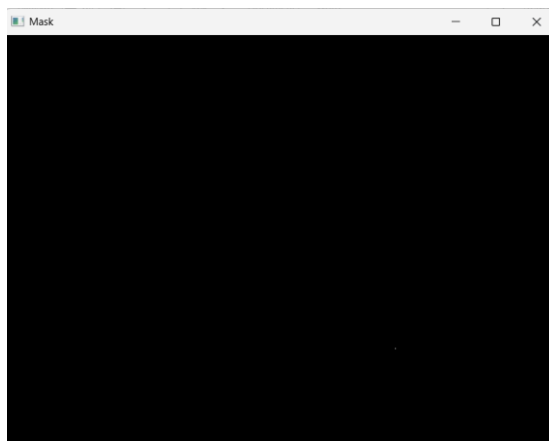
Purpose: To automatically segment and isolate the main object from its background.

Code 14:

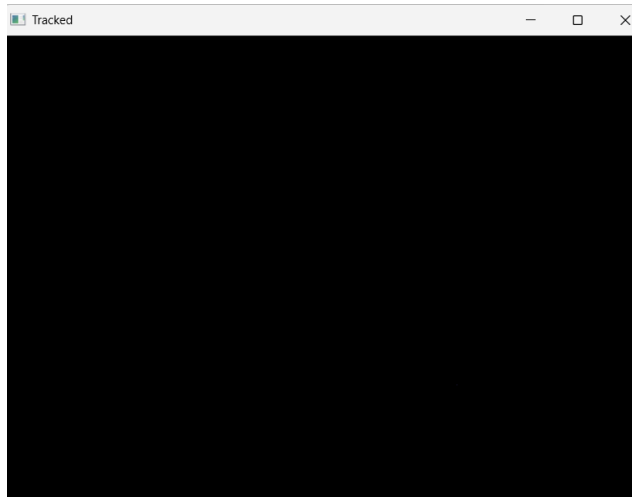
Input: The input for this program is the live video stream captured from the system's webcam.

Output:

The output displays the live video frame from the webcam along with the generated mask window, which highlights only the detected blue color regions (black if no blue color is present).



Mask



Tracked

Logic:

1. Opens the webcam using `cv2.VideoCapture(0)`.
2. Continuously reads each video frame.
3. Converts the frame to HSV color space (good for color detection).
4. Defines the HSV range for blue color.
5. Creates a mask that highlights only blue areas.
6. Uses `bitwise_and()` to extract and display only the blue-colored regions.
7. Displays the original frame, mask, and tracked (blue-only) output.
8. Stops when the 'q' key is pressed.

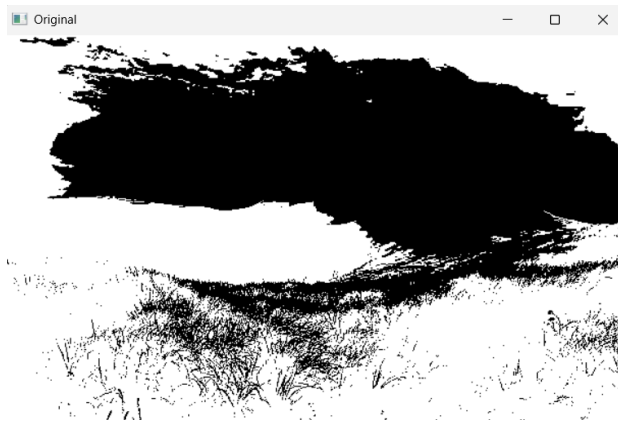
Purpose: To track blue objects live from a webcam feed using color detection.

Code 15:

Input:



Output:



original



Erosion



Dilation

Logic:

1. Reads the image in grayscale.
2. Applies binary inverse thresholding to convert it into black and white.
3. Creates a 5×5 kernel (structuring element).
4. Erosion removes small white noises and shrinks white regions.
5. Dilation enlarges white areas and fills small holes.
6. Displays the thresholded, eroded, and dilated images.

Code 16:

Input:

s = "Emails like help@site.com are filtered. I'm LOVING NLP!!! Visit <https://x.y>."

Output:

```
['email', 'like', 'filter', 'love', 'nlp', 'visit']
```

Logic:

1. `basic_clean()` → Converts text to lowercase, removes URLs, emails, mentions, hashtags, and special characters, keeping only letters, numbers, spaces, and apostrophes.

2. `tokenize_stop_lemma()` →

Uses spaCy to tokenize the cleaned text.

Converts words to their lemmas (base forms).

Removes stop words, punctuation, and short tokens (less than 3 characters).

3. `preprocess()` → Combines cleaning and tokenization steps to return a final list of meaningful words.

4. When run, it processes a sample sentence and prints the cleaned, lemmatized word list.

Purpose: To prepare raw text for NLP or machine learning by cleaning, tokenizing, lemmatizing, and removing unnecessary words.

Code 17:

Input:

The input for this program is a small dataset of text sentences containing positive and negative movie reviews used for training and testing a sentiment classification model.

Output:

```
Classification report:
              precision    recall  f1-score   support

     0       0.0000      0.0000      0.0000         2
     1       0.3333      1.0000      0.5000         1

   accuracy          0.3333         3
  macro avg       0.1667      0.5000      0.2500         3
 weighted avg       0.1111      0.3333      0.1667         3

Confusion matrix:
[[0 2]
 [0 1]]
Predictions: [1 1]
Class probabilities: [[0.42652933 0.57347067]
 [0.46719711 0.53280289]]
```

Logic:

1. Creates a small dataset of movie reviews labeled as positive (1) or negative (0).
2. Splits data into training (70%) and testing (30%) sets.
3. Builds a pipeline:

TfidfVectorizer → converts text into numerical features using TF-IDF (word importance).

LogisticRegression → learns to classify reviews as positive or negative.

4. Trains (fit) the model on the training data.

5. Predicts labels on the test data and prints:

Classification report (precision, recall, f1-score).

Confusion matrix (correct vs. wrong predictions).

6. Tests the model on new sample reviews and shows predicted sentiment and class probabilities.

Purpose: To automatically classify text reviews as positive or negative using machine learning.

Code 18:

Input:

The input for this program is a small set of movie review sentences along with their sentiment labels (positive or negative) used for training and tuning the text classification model.

Output:

```
Best params: {'clf__C': 4.0, 'tfidf__analyzer': 'char_wb', 'tfidf__min_df': 1, 'tfidf__ngram_range': (1, 2)}
Best CV score (f1): 0.7222222222222222
Sample prediction: [1]
```

Logic:

1. A small dataset of movie reviews is created with labels — 1 (positive) and 0 (negative).
2. A pipeline is built with two steps:

TfidfVectorizer → converts text to numerical TF-IDF features.

LogisticRegression → classifies text sentiment.

3. A parameter grid (param_grid) defines combinations of tuning options:

Different ngram ranges (single words or word pairs).

Minimum word frequency (min_df).

Feature extraction method (word vs. char_wb).

Logistic Regression's regularization strength (C).

4. GridSearchCV tests all parameter combinations using 3-fold cross-validation and selects the best based on F1-score.

5. Prints the best parameters, best cross-validation score, and uses the best model to predict sentiment for a new sample sentence.

Purpose: To automatically find the most effective settings for a text sentiment classifier using grid search and cross-validation.

Code 19:

Input:

The input for this program is a set of sample text documents containing sentences related to machine learning and artificial intelligence.

Output:

```
Vocabulary: ['advances' 'and' 'artificial' 'deep' 'fun' 'intelligence' 'is' 'learning'
'machine']
```

```
TF-IDF Matrix:
```

	advances	and	artificial	deep	fun	intelligence	is	learning	\
0	0.000	0.000	0.000	0.000	0.609	0.00	0.609	0.360	
1	0.552	0.000	0.000	0.552	0.000	0.42	0.000	0.326	
2	0.000	0.552	0.552	0.000	0.000	0.42	0.000	0.326	

	machine
0	0.360
1	0.326
2	0.326

Logic:

1. Defines a small list of text documents.

2. Uses TfidfVectorizer to:

Learn the vocabulary (all unique words).

Compute TF-IDF scores for each word in each document (importance of a word relative to the document and corpus).

3. Converts the resulting sparse matrix into a Pandas DataFrame for easy viewing.

4. Prints the vocabulary and the TF-IDF values for each document.

Purpose: To numerically represent text for machine learning models by showing how important each word is in the documents.

Code 20:

The input for this program is a text paragraph containing sentences about Apple's new office and meetings, which is processed for named entities, POS tags, and noun chunks.

Output:

```
Named Entities (text, label):
```

```
Apple          -> ORG
Bengaluru      -> GPE
next quarter   -> DATE
Tim Cook       -> PERSON
Karnataka      -> GPE
September 3, 2025 -> DATE
```

```
Part-of-Speech & Lemmas:
```

```
Apple      POS=PROPN  Lemma=Apple
is          POS=AUX    Lemma=be
opening     POS=VERB   Lemma=open
a           POS=DET    Lemma=a
new         POS=ADJ    Lemma=new
office      POS=NOUN   Lemma=office
in          POS=ADP    Lemma=in
Bengaluru   POS=PROPN  Lemma=Bengaluru
next        POS=ADJ    Lemma=next
quarter     POS=NOUN   Lemma=quarter
.           POS=PUNCT  Lemma=.
Tim         POS=PROPN  Lemma=Tim
Cook        POS=PROPN  Lemma=Cook
met         POS=VERB   Lemma=meet
Karnataka   POS=PROPN  Lemma=Karnataka
officials    POS=NOUN   Lemma=official
on          POS=ADP    Lemma=on
September   POS=PROPN  Lemma=September
3           POS=NUM    Lemma=3
,           POS=PUNCT  Lemma=,
2025        POS=NUM    Lemma=2025
to          POS=PART   Lemma=to
discuss     POS=VERB   Lemma=discuss
expansion   POS=NOUN   Lemma=expansion
.           POS=PUNCT  Lemma=.
```

```
Noun chunks (base NPs):
```

```
['Apple', 'a new office', 'Bengaluru', 'Tim Cook', 'Karnataka officials', 'September', 'expansion']
```

Logic:

1. Loads the small English model (en_core_web_sm) in spaCy.
2. Processes the text to create a Doc object that contains linguistic annotations.
3. Named Entity Recognition (NER):

Extracts entities like organizations, dates, locations, and prints their text and type.

4. Part-of-Speech (POS) tagging & Lemmatization:

Prints each token's POS tag (noun, verb, etc.) and lemma (base form).

5. Noun Chunks:

Extracts base noun phrases (grouped nouns with modifiers) from the text.

Purpose: To analyze text structure, recognize entities, understand grammar, and extract meaningful phrases for NLP tasks.

Code 21:

Input:

The input for this program is a small set of movie review sentences along with their sentiment labels (positive or negative) used for training and testing the Naive Bayes classifier.

Output:

```
Classification Report:
              precision    recall  f1-score   support

     0       0.50         1.00         0.67         1
     1       0.00         0.00         0.00         1

 accuracy          0.50         2
 macro avg         0.25         0.50         0.33         2
 weighted avg      0.25         0.50         0.33         2
```

Logic:

1. Defines a small dataset of movie reviews labeled as positive (1) or negative (0).
2. Splits data into training (70%) and testing (30%) sets.
3. Uses CountVectorizer to convert text into Bag-of-Words representation (counts of each word in the vocabulary).
4. Trains a Multinomial Naive Bayes classifier on the BoW features from training data.
5. Predicts labels for the test data.
6. Prints a classification report showing precision, recall, f1-score, and accuracy.

Purpose: To classify text reviews as positive or negative based on word frequencies using a simple probabilistic model.

Code 22:

Input:

The input for this program is a set of example text documents used to compute TF-IDF features and cosine similarity between them.

Output:

```
Cosine Similarity Matrix:
[[1.          0.64424596 0.          ]
 [0.64424596 1.          0.12413287]
 [0.          0.12413287 1.          ]]
```

Logic:

1. Defines a list of documents.
2. Uses TfidfVectorizer to convert documents into TF-IDF vectors, capturing the importance of words in each document.
3. Computes cosine similarity between all pairs of TF-IDF vectors, which measures how similar two documents are based on content.
4. Prints the similarity matrix, where values range from 0 (no similarity) to 1 (identical).

Purpose: To quantitatively measure how similar text documents are for tasks like information retrieval, clustering, or plagiarism detection.

Code 23:**Input:**

The input for this program is a set of example text documents used for tokenization, dictionary creation, and topic modeling using LDA.

Output:

```
Topic 0: 0.072*"i" + 0.071*"the" + 0.043*"trying" + 0.043*"new" + 0.043*"recipes" + 0.043*"kitchen" + 0.043*"processing" + 0.043*"enjoy" + 0.043*"in" + 0.043*"deep"
Topic 1: 0.094*"and" + 0.092*"are" + 0.056*"learning" + 0.055*"machine" + 0.055*"closely" + 0.055*"ai" + 0.055*"related" + 0.055*"cooking" + 0.055*"hobbies" + 0.055*"baking"
```

Logic:

1. Defines a list of documents.
2. Tokenizes each document into lowercase words.
3. Creates a dictionary (mapping of unique words to IDs) and a corpus (bag-of-words representation for each document).
4. Trains an LDA model to discover 2 latent topics from the corpus (passes=10 controls training iterations).
5. Prints the topics, showing the most important words for each topic.

Purpose: To identify underlying topics in a collection of text documents automatically.